* Merge two sorted arrays

* Two pointer approach

i = 0
j = 0
k = 0

a = {1, 3, 5, 7, 9} copy

b = {2, 4, 6} out of array

if a[i] < b[j]
  c[k++] = a[i++]
else b[j] < a[i]
  c[k++] = b[j++]

c = {1, 2, 3, 4, 5, 6, 7, 9} paste

* Till the length of the two arrays are equal compare & store in the final array.

* If any array has extra elements, they are already sorted. Just copy / paste in the final array.

Merge Sort Algorithm :→ Divide & Conquer Algorithm

(Recursion)

split = log n
merge = n
total = n log n
S/C = O(n)

Single Element (Sorted Array)

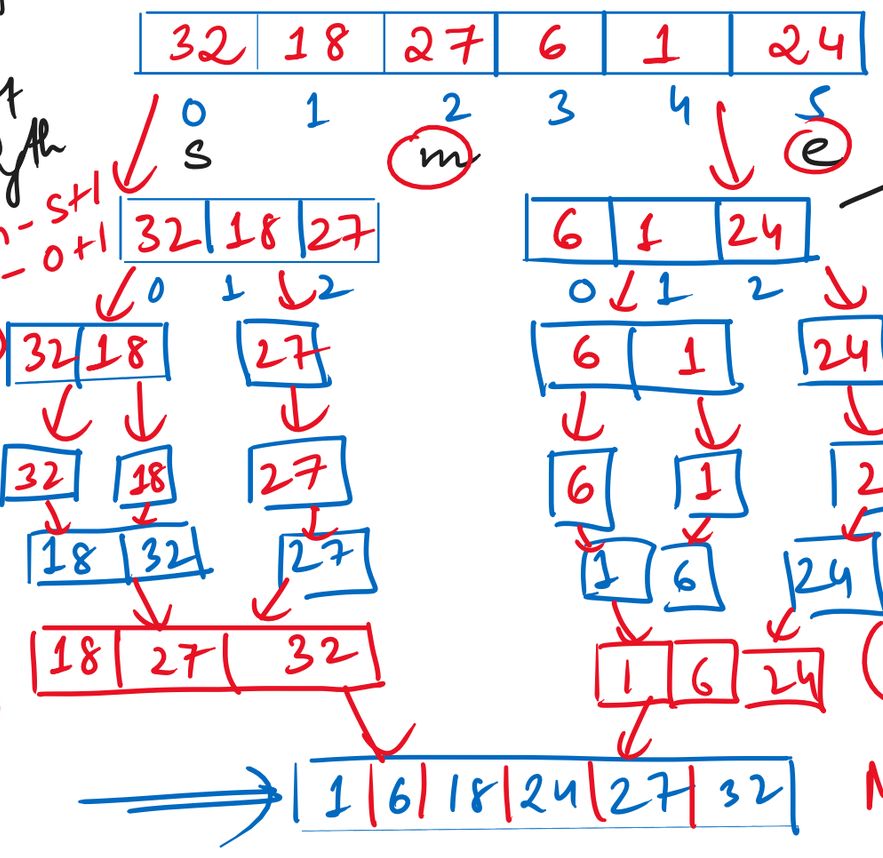
left length = m - s + 1 = 2 - 0 + 1 = 3

right length = e - m = 5 - 2 = 3

m = (s + e)/2 = s/2

log n = 2

(Split)



(Left sorted array)

(Right Sorted Array)

Merging two sorted arrays.

n × log n

* Quick Sort Algorithm :→

→ This algorithm works with the intuition of finding a partition index & a pivot element in the given array.

→ The purpose of finding the pivot element is :→

a) Any element less than pivot goes left.

b) Any element greater than pivot goes right.

{pivot = 5}

n log n (Recursion)

variables pi | pe

arr → (5, 9, 1, 0, 4, 6, 3) s, e

① pivot = arr[s] = 5

② We find it's correct position
count = 0; ++ ++ ++ ++ 4

③ We find out how many elements are less than the pivot.

⑤ partition index = s + count = 0 + 4 = 4

⑥ Swap s & pi values

(4, 9, 1, 0, 5, 6, 3)
s, i, pi, e

i = s
j = e
while (arr[i] < pivot) i++
while (arr[j] > pivot) j-- (Recursion)

4 3 1 0 5 6 9
i i j j

if values are at wrong places (swap)